

Logic Optimization Meets SAT: A Novel Framework for Circuit-SAT Solving

Zhengyuan Shi^{1,4}, Tiebing Tang¹, Jiaying Zhu^{1,4}, Sadaf Khan¹,
Hui-Ling Zhen², Mingxuan Yuan², Zhufei Chu³ and Qiang Xu^{1,4}

¹Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong S.A.R.

²Noah's Ark Lab, Huawei, Hong Kong S.A.R.

³Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo, China

⁴National Center of Technology Innovation for EDA, Nanjing, China

Abstract—The Circuit Satisfiability (CSAT) problem, a variant of the Boolean Satisfiability (SAT) problem, plays a critical role in integrated circuit design and verification. However, existing SAT solvers, optimized for Conjunctive Normal Form (CNF), often struggle with the intrinsic complexity of circuit structures when directly applied to CSAT instances. To address this challenge, we propose a novel preprocessing framework that leverages advanced logic synthesis techniques and a reinforcement learning (RL) agent to optimize CSAT problem instances. The framework introduces a cost-customized Look-Up Table (LUT) mapping strategy that prioritizes solving efficiency, effectively transforming circuits into simplified forms tailored for SAT solvers. Our method achieves significant runtime reductions across diverse industrial-scale CSAT benchmarks, seamlessly integrating with state-of-the-art SAT solvers. Extensive experimental evaluations demonstrate up to 63% reduction in solving time compared to conventional approaches, highlighting the potential of EDA-driven innovations to advance SAT-solving capabilities.

I. INTRODUCTION

The Circuit Satisfiability (CSAT) problem lies at the heart of many critical tasks in Electronic Design Automation (EDA), such as logic equivalence checking (LEC), automatic test pattern generation (ATPG), and logic synthesis (LS). It involves determining whether a Boolean circuit has an input assignment that satisfies its primary output. As a specialized variant of the Boolean Satisfiability (SAT) problem, CSAT is known to be NP-complete [1], making it both theoretically challenging and practically important in integrated circuit (IC) design and verification.

Modern SAT solvers, such as MiniSAT, Kissat, and CaDiCaL [2], [3], have been highly effective in solving problems represented in Conjunctive Normal Form (CNF). However, when applied to CSAT problems, these solvers face inefficiencies stemming from the required transformation of circuits into CNF format. This transformation often disrupts the natural structure of circuits, leading to representations that are less solver-friendly and more complex to process [4], [5]. As a result, solving CSAT instances may become unnecessarily time-consuming, particularly for large-scale, industrial circuits.

While preprocessing techniques for CNF have been extensively explored [4]–[7], similar advancements tailored specifically for circuit-based representations remain underdeveloped.

Circuit-specific methods often focus on reducing circuit size or area [8], which, while useful for design optimization, do not directly address the complexity of SAT solving. This gap highlights the need for a new approach that bridges the structural strengths of circuit representations with the solving power of modern SAT solvers.

To address these limitations, we propose a novel preprocessing framework tailored to CSAT problems. Our approach integrates EDA techniques with reinforcement learning (RL) to transform CSAT instances into SAT-friendly formats. By leveraging logic synthesis tools and a cost-customized Look-Up Table (LUT) mapping strategy, our framework achieves significant reductions in solving complexity. Unlike traditional preprocessing methods, our approach explicitly prioritizes SAT-solving efficiency rather than widely-used optimization metrics like area or delay. Specifically, the main contributions of our work include:

- **EDA-Driven Preprocessing Framework:** We introduce a comprehensive preprocessing pipeline that combines logic synthesis techniques with reinforcement learning to optimize CSAT instances for efficient SAT solving. This integration bridges the gap between raw circuit structures and modern SAT solvers.
- **RL-Guided Logic Synthesis for SAT Solving:** We formulate logic synthesis as a Markov Decision Process (MDP) and train an RL agent to identify synthesis sequences that minimize solving complexity. The agent leverages circuit features inspired by [9], [10] and structural circuit embeddings from DeepGate2 [11].
- **Cost-Customized LUT Mapping for SAT Solving:** We develop a novel LUT mapping strategy that incorporates a customized cost metric reflecting branching complexity during SAT solving. This approach diverges from conventional LUT mapping methods [12], [13] by focusing on solving efficiency.

Using industrial-scale benchmarks derived from LEC and ATPG tasks, we demonstrate significant runtime reductions up to 63% compared to baseline approaches. Our method integrates seamlessly with state-of-the-art SAT solvers, such as Kissat and CaDiCaL.

Corresponding author: Qiang Xu (qxu@cse.cuhk.edu.hk)

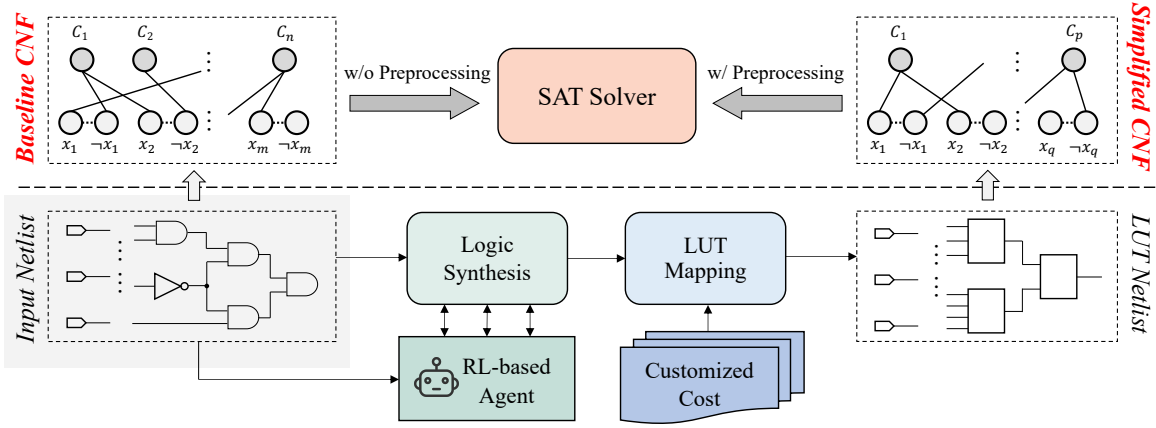


Fig. 1: Overview of EDA-Driven SAT preprocessing framework

II. RELATED WORK

A. SAT and Circuit SAT Problems

SAT problem holds a pivotal position in hardware design, software verification, planning and scheduling, which seeks to identify if there exists at least one assignment that makes a given Boolean formula True. The modern SAT solvers, central in addressing SAT problems, predominantly rely on the Conflict-Driven Clause Learning (CDCL) algorithm [14]. Within the backbone CDCL algorithm, CNF has become a standard format in SAT solving, capable of representing arbitrary Boolean formulas. It is structured as a conjunction of clauses, denoted by $\phi = (C_1 \wedge C_2 \wedge \dots)$, with each clause being a disjunction of variables or their negations, e.g., $C_i = (x_1 \vee \neg x_2 \vee \dots)$. Over recent decades, numerous CNF-based heuristics have been developed to enhance SAT solving efficiency, including advanced clause management strategies [15] and innovative branching heuristics [16], [17].

Typically, practical CNF-modeled problems often result in redundant clauses and increased complexity. For example, a tournament scheduling problem with n teams can escalate to n^6 clauses [18]. Consequently, problem reformulation, as known as preprocessing, is essential before solving. Efforts like [4]–[6] target minimizing the CNF instances by resolution and graph transformation, resulting in faster SAT solving.

One of the most representative SAT variants in the realm of EDA is the Circuit SAT (CSAT) problem. In contrast to the general SAT problem, the CSAT problem determines the satisfiability of a Boolean circuit. Over the past decades, circuit-based SAT solvers have been developed to tackle the CSAT problems [19]–[21]. Despite these solvers conducting logic reasoning in natural formats, they still exhibit lower performance levels compared to CNF-based SAT solvers due to the inefficient circuit-based preprocessing and heuristics. For example, the previous circuit-based preprocessing approaches, such as [8], cannot promise to reduce solving time as they only scale down circuits. In this work, we introduce an efficient and effective circuit-based preprocessing framework to directly mitigate solving complexity by integrating EDA tools and reinforcement learning guidance.

B. Boolean Circuit Optimization

Boolean circuits, representable as directed acyclic graphs (DAGs) $\mathcal{G}(\mathcal{V}, \mathcal{E})$ with logic gates and wires serving as nodes $\mathcal{V}$ and edges $\mathcal{E}$, respectively, are pivotal in computational logic. In the EDA community, a wealth of efficient algorithms has been developed for circuit optimization. Logic Synthesis (LS) is a prime example, involving DAG-based transformation techniques to modify circuit's local structure for the optimization of performance, power, and area (PPA).

Besides, technology mapping, a key component in EDA flow, fine-tunes performance metrics under specific constraints [22] and maps the intermediate circuits into standard cells pre-defined in technology library. One widely used technique for technology mapping is Look-Up Table (LUT) mapping [12]. LUT mapping involves the transformation of gate combinations into k -LUTs, where each k -LUT represents a configurable memory unit capable of implementing arbitrary Boolean functions with k variables.

The advent of circuit representation learning [23] marks a progressive trend in the field, focusing on deriving neural representations of circuits to facilitate downstream tasks. The DeepGate family [11], [24], [25], for instance, integrates structural and functional aspects of circuits and has shown impressive results in testability analysis [26] and SAT solving [21]. In our research, we tailor existing EDA tools for specific requirements, utilizing DeepGate2's [11] circuit knowledge learning capabilities to inform our circuit optimization decisions.

III. METHODOLOGY

A. Overview of the Proposed Framework

Fig. 1 shows the overview of the proposed circuit-based preprocessing framework. Given a CSAT problem represented in And-Inverter Graphs (AIGs), which only contain three gate types (primary input, AND and NOT) by [27], the baseline pipeline converts it into CNF.

In our proposed framework, we first apply a predetermined sequence of AIG transformation operations to unify the distribution of input circuits [21]. Then, we utilize logic synthesis technique to reformulate SAT instance. Since the

logic synthesis process can be considered as a sequential decision-making task, we train an agent to minimize the solving complexity by reinforcement learning (RL). In the RL framework, the state comprises embeddings of the initial AIG netlist and features of intermediate circuits. Additionally, the RL agent operates within a discrete action space, where each action corresponds to executing a synthesis operation. We approximate the complexity of SAT solving based on the number of variable branching times and define the reduction in branching times as the reward for RL agent.

Finally, we employ the LUT mapping technique to hide the internal logic of circuits. To further decrease the solving time, we design a cost-customized mapping operation with the objective of minimizing the branching times during SAT solving, rather than solely focusing on reducing the problem size. After the LUT mapping, we obtain the LUT netlist with a relatively small number of cells. To ensure compatibility with modern SAT solvers, we transform the LUT netlist back into a simplified CNF.

B. Logic Synthesis Recipe Exploration

We formulate LS as a Markov Decision Process (MDP) process, with the optimization objective of reducing the solving time for a given input instance. Consequently, such sequential decision-making process entails iteratively selecting a LS operation as an action according to the current circuit state.

1) *Reinforcement Learning Formulation*: To explore the optimal LS sequence (i.e. LS recipe), we employ Deep Q-learning algorithm in this section to train a reinforcement learning (RL) agent. The RL agent is trained to make informed decisions to minimize solving time by learning from its interactions with the circuit environment.

As shown in Fig. 2, at each step t (where $t = 0, 1, \dots, T-1$), the features of the current netlist $\mathcal{G}^t$ are extracted to form the state s^t . Subsequently, the Q-learning agent utilizes s^t as input and selects an action a^t . Then, the environment including the LS tool performs the synthesis operation based on a^t and transforms the netlist $\mathcal{G}^t$ into a new netlist $\mathcal{G}^{t+1}$. At the end of this step, the environment provides RL agent with a reward r^t . The above process is repeated for subsequent steps until $t = T-1$. During the training process, the objective of RL agent is to maximize the cumulative sum of the reward values. As a result, the trained RL agent is able to select LS operations step by step to achieve the optimal solving time reduction.

2) *State*: Inspired by [9], [10], we extract the following representative features $\mathcal{E}(\mathcal{G}^t)$ of the netlist $\mathcal{G}^t$ into state s^t :

- Ratio between the area of netlist $\mathcal{G}^t$ and netlist $\mathcal{G}^0$.
- Ratio between the depth of netlist $\mathcal{G}^t$ and netlist $\mathcal{G}^0$.
- Ratio between the wire counts of netlist $\mathcal{G}^t$ and netlist $\mathcal{G}^0$.
- Proportion of AND gates in the total gates of netlist $\mathcal{G}^t$.
- Proportion of NOT gates in the total gates of netlist $\mathcal{G}^t$.
- Average balance ratio of netlist $\mathcal{G}^t$. We formally define the average balance ratio br in Eq. (1), where $\mathcal{P}_{i1}$, $\mathcal{P}_{i2}$ are predecessors of two fanin AND gate i , and d denotes the depth.

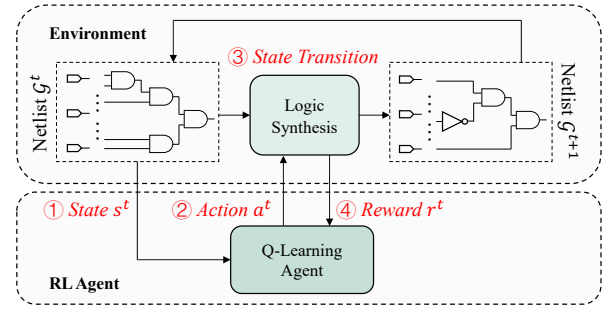


Fig. 2: RL Agent-Environment interaction

$$br = \sum_{i \in AND} \frac{abs(d_{\mathcal{P}_{i1}} - d_{\mathcal{P}_{i2}})}{max(d_{\mathcal{P}_{i1}}, d_{\mathcal{P}_{i2}})} / \#AND \quad (1)$$

The above features exhibit characteristics that can be effectively optimized by various logic synthesis operations. Consequently, the RL agent can intelligently select appropriate operations based on these features. For example, when a netlist shows a higher average balance ratio, indicating an imbalanced structure in DAG, the agent is more inclined to select the action of operation *balance*. Additionally, a large area ratio prompts the agent to opt for operations that aim to minimize the circuit area, such as *rewrite*.

Besides, the state s^t is incorporated with the primary output embeddings of the initial netlist obtained by pretrained DeepGate2 [11] (denoted as $\mathcal{D}(\mathcal{G}^0)$), which contains rich structural and functional information of the initial problem instance.

$$s^t = \text{Concatenate}(\mathcal{E}(\mathcal{G}^t), \mathcal{D}(\mathcal{G}^0)) \quad (2)$$

3) *Action*: The action space $\mathcal{A}$ of agent is discrete and encompasses LS operations. In this paper, we set the available operations as follows: *rewrite* [28], *refactor* [29], *balance* [30], *resub* [31] and *end* (marking the end of the LS process). The selection of these operations does not imply that the proposed framework lacks support for additional synthesis operations beyond this set. Instead, these operations are chosen due to their widespread usage and commonality. If there is a need to include additional optional operations, it can be accomplished by expanding the action space within the framework.

4) *State Transition*: The state transition function is implemented by the LS tool. We call the tool with a specific LS operation determined by RL agent to transform the current netlist $\mathcal{G}^t$ to another functional-equivalent but simplified netlist. More formally, we note the given netlist as $\mathcal{G}^t$ and RL action as a^t at step t . The state transition function is denoted as: $\mathcal{G}^{t+1} = \mathcal{F}(\mathcal{G}^t, a^t)$, where $\mathcal{G}^{t+1}$ is the updated netlist by tool. The updated netlist, $\mathcal{G}^{t+1}$, is then considered as the input for the subsequent decision step at time $t+1$.

5) *Reward*: We consider the reduction of variable branching times during SAT solving as the reward. The reward function is shown as Eq. (3), where $\Delta\#Branching$ is the difference in branching times between the final instance and the initial instance during solving. We opt for the terminated reward, which has only one non-zero reward value at the

terminated step, i.e. after performing the entire sequence of synthesis operations.

$$r^t = \begin{cases} 0, t = 0, \dots, T-2, \\ -\Delta \# \text{Branching}, t = T-1 \text{ or } a^t = \text{end} \end{cases} \quad (3)$$

Prior to solving the SAT instance in circuit format, we employ the LUT mapping to convert the AIG netlist into an LUT netlist. Subsequently, the transformed netlist is converted into CNF, which will be elaborated in the following section. Additionally, we choose not to minimize the solver runtime based on two primary reasons. Firstly, determining the solver runtime for hard SAT instances is time-consuming. Secondly, the solver runtime is challenging to calculate accurately due to the fluctuating CPU status in the case of easy SAT instances. We utilize variable branching times to approximate solving time. This approach allows us to include easy cases with short solver runtime in the training dataset, thereby enhancing the efficiency of training the RL agent.

6) *Policy*: In the Q-learning framework [32], the policy for maximizing the total reward entails selecting the action with the highest expected value. The action value function Q uses the multilayer perceptron (MLP) to estimate the sum of future rewards after taking a certain action in the current state. The policy $\pi(s^t)$ and action-value function $Q_\theta(s^t, a)$ are shown in Eq. (4), where the θ parametrizes the function Q .

$$\begin{aligned} \pi(s^t) &= \arg \max_{a, a \in \mathcal{A}} (Q(s^t, a)) \\ Q_\theta(s^t, a) &= \text{Index}(\text{MLP}(s^t), a) \end{aligned} \quad (4)$$

We create the target function $Q_{\hat{\theta}}(s^t, a)$ to stabilize the training process. The parameter $\hat{\theta}$ is copied from θ after k steps. The parameter θ of action-value function Q is updated iteratively by loss function in Eq. (5), where γ is a hyperparameter and represents discount factor.

$$l = \|Q_\theta(s^t, a^t) - r^t - \gamma \arg \max_{a \in \mathcal{A}} Q_{\hat{\theta}}(s^{t+1}, a)\|_2 \quad (5)$$

C. Cost-Customized LUT Mapping

We present a cost-customized LUT mapping approach that not only hides the internal nodes during solving, but also forms an easy-to-solve instance. Unlike conventional mapping tools that focus on constructing circuits with low area or delay, our approach aims to create circuits that are easier to solve. Specifically, we begin by defining a cost metric to quantify the branching complexity of SAT solving and compute the cost values for all 4-input LUTs. Next, we modify the cost function in the existing mapping tool to enable minimizing the overall branching complexity. Lastly, we apply the cost-customized mapper to map AIG netlist to LUT netlist and convert the LUT netlist into CNF.

1) *Branching Complexity of LUT*: The SAT solving process involves iteratively making variable branching decisions, where the solver selects variable assignments as either *true* or *false*. We formulate the solving process within the circuit domain. Specifically, given the logic value of a gate, the SAT solver makes decisions by selecting assignments for its

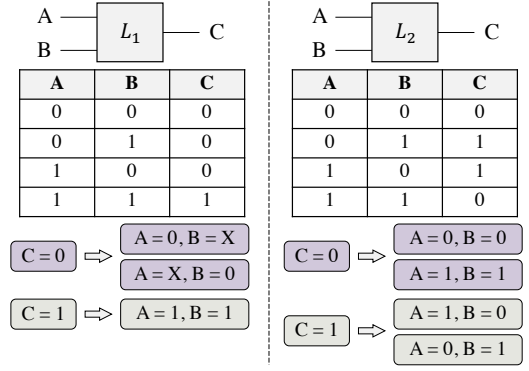


Fig. 3: LUTs with different branching complexity

fanin gates. We consider two 2-fanin LUTs, denoted as L_1 (representing an AND gate) and L_2 (representing an XOR gate), and list the truth tables for these LUTs in Fig. 3.

We observe that both L_1 and L_2 have two possible combinations when the fanout pin (C) is logic-0. However, when the fanout pin is logic-1, L_2 has two possible combinations while L_1 has only one unique branch. We define the *branching complexity* C as the total number of possible fanin combinations. In this example, the LUT L_1 has $C_{L_1} = 3$ and LUT L_2 has $C_{L_2} = 4$. Therefore, L_2 is more hard to be solved than L_1 .

2) *Cost-Customized Mapper*: Since a large number of possible branches result in an exponential search space, various heuristics [14], [33] are employed to reduce the number of potential branches and improve the efficiency of the solving process. Besides, utilizing technique mapping tools facilitates the concealment of intermediate gates and decreases the overall number of branches [8].

In contrast to [8] that minimizes the size of the post-mapping netlist, our approach prioritizes mapping the logic into cells with lower branching complexity. Our approach is motivated by the observations that instances containing a significant proportion of XOR gates often require more time to be solved [34], [35], where the branching complexity of XOR gate (L_2 in Fig. 3) is higher than other gates.

To prioritize the LUTs during mapping process, we employ a strategy to enumerate all 4-LUTs and integrate their branching complexity into the cost function. We implement a cost-customized mapper based on mockturtle [13] by modifying the area cost to reflect the branching complexity of each LUT and fixing the delay cost as a constraint. By executing a sequence of operations aimed at minimizing the total cost, the resulting post-mapping netlists exhibit minimal overall branching complexity. Our preprocessing strategy ends with a LUT to CNF transformation [36] for compatibility.

As a result, we formally depict our overall circuit-based preprocessing framework coupled with RL-based logic synthesis exploration and cost-customized LUT mapping in Algorithm 1. The framework begins by transforming the input circuit instance $\mathcal{G}_{in}$ into AIG format, using *aigmap* technique in ABC [37]. Then, the RL agent (elaborated in Section III-B) proceeds to select logic synthesis operations step by step through making action a^t . During this process (line 6-16),

Algorithm 1: CSAT preprocessing framework

Input: The input circuit instance $\mathcal{G}_{in}$
Output: The output simplified CNF instance ϕ_{out}
 /* Initialize circuit graph $\mathcal{G}^0$ in AIG
 */
 1 **if** *input instance is in AIG* **then**
 2 $\mathcal{G}^0 = \mathcal{G}_{in}$;
 3 **else**
 4 $\mathcal{G}^0 = \text{aigmap}(\mathcal{G}_{in})$;
 5 **end**
 /* Explore logic synthesis recipe by
 RL-agent (see Section III-B) */
 6 Define maximum step count: T ;
 7 $t = 0$;
 8 **while** $t < T$ **do**
 /* Prepare the state of current
 circuit */
 9 $s^t = \text{Concatenate}(\mathcal{E}(\mathcal{G}^t), \mathcal{D}(\mathcal{G}^0))$;
 /* Make decision to choose a logic
 synthesis operation */
 10 $a^t = \pi(s^t) = \arg \max_{a, a \in \mathcal{A}} (Q(s^t, a))$;
 11 **if** a^t *is end* **then**
 12 **break**;
 13 **end**
 /* Perform logic synthesis for
 state transition */
 14 $\mathcal{G}^{t+1} = \mathcal{F}(\mathcal{G}^t, a^t)$;
 15 $t = t + 1$;
 16 **end**
 /* Perform cost-customized LUT
 mapping (see Section III-C) */
 17 Define LUT costs: C ;
 18 $\mathcal{G}_{LUT} = \text{mapper}(\mathcal{G}^t, C)$;
 /* Convert to CNF */
 19 $\phi_{out} = \text{lut2cnf}(\mathcal{G}_{LUT})$;
 20 **return** ϕ_{out} ;

the circuit is synthesized multiple times to create a simplified circuit. Next, the framework continues to perform cost-customized LUT mapping on the AIG netlist $\mathcal{G}^t$, resulting in a compact LUT netlist $\mathcal{G}$. Finally, the LUT netlist is encoded into CNF as output ϕ_{out} . Due to these simplification approaches, ϕ_{out} has a lower solving complexity than ϕ_{in} and the subsequent SAT solving procedures can be more efficient. We will assess the effectiveness of our preprocessing by the following experiments.

IV. EXPERIMENTS

A. Experiment Settings

We collect 200 easy (solving time 0.04s – 6.68s) instances into the RL training dataset and 300 hard instances for testing. All instances are in AIG format and derived from both industrial logic equivalence checking (LEC) and automatic test pattern generation (ATPG) problems. For LEC instances,

TABLE I: Statistics of training dataset

	Avg.	Std.	Min.	Max.
# Gates	4,299.06	4,328.16	60	24,178
# PIs	43.66	25.17	6	102
Depth	66.43	19.98	18	138
# Clauses	10,687.28	10,801.96	131	60,294
Time (s)	2.01	1.96	0.04	6.68

we modify original industrial datapath circuits and connect their primary outputs (POs) through XOR gates to form CSAT instances, where satisfiability indicates non-equivalent functionality between circuits. For ATPG instances, we introduce stuck-at faults into industrial circuits and connect the POs of faulty and fault-free circuits through XOR gates, where satisfiable assignments serve as test patterns for fault detection. We construct 200 LEC instances and 100 ATPG instances in total, where 200 easy instances are selected for RL training based on their clause count after CNF transformation and solving time without preprocessing. The statistics of training instances are shown in Table I, where # Clauses shows the number of clauses after transforming AIG instances into CNFs and Time represents the solving time without any preprocessing.

In the RL setting, we assign the maximum number of steps per episode as $T = 10$ and the discount factor $\gamma = 0.98$. During each training episode, the agent randomly selects an instance and makes decisions aimed at transforming the instance to minimize the number of branching times in SAT solving. We conduct a total of 10,000 episodes, i.e., the RL agent preprocesses instances for 10,000 times during RL training. The batch size is 32 and learning rate is 10^{-5} .

We opt for the ABC tool [37] to synthesize the circuit and build a cost-customized mapper based on mockturtle [13] to map the circuit in LUT netlists. The solver runtime limitation is 1,000s. Any cases that exceed this time limit are marked with TO (Time-out) and are considered to have a solving time of 1,000 seconds ($\mathcal{T}_{solve} = 1,000s$). Each instance solving is conducted on a single core of Intel Xeon CPU E5-2630.

B. Runtime Comparison

Figure 4 illustrates the relationship between the number of instances solved and overall runtime (including RL model inference time, transformation time and solving time). The Baseline setting represents the conventional solving pipeline, encoding the circuit-based instances directly into CNFs. Our experiment evaluates the effectiveness of our proposed framework using both Kissat solver (version 4.0.0 [38]) and CaDiCaL solver (version 2.0.0 [39]), with our specific settings distinguished by the orange line (denoted as Ours).

To showcase the efficiency of our preprocessing framework, we compare the solving time with another circuit-based approach¹ [8], denoted as Comp. . To the best of our knowledge, although such approach has been around for over a decade, this is the only circuit-based SAT preprocessing technique.

¹We do not include comparisons with CNF-based preprocessing approaches because our framework is not mutually exclusive with them and can be used in conjunction. Practically, we keep the default CNF-based preprocessing.

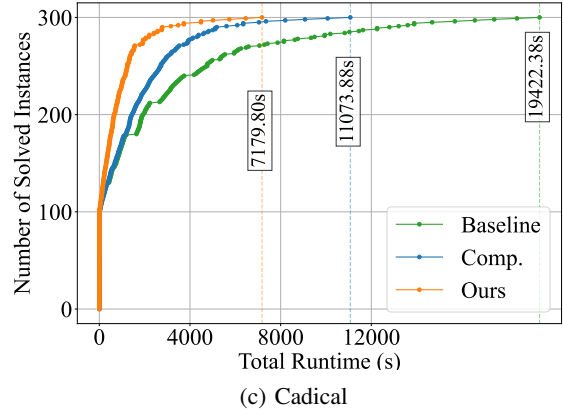
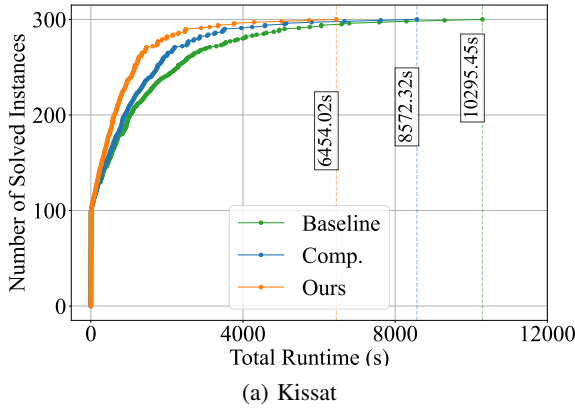


Fig. 4: Runtime Comparison with Baseline and Comp.

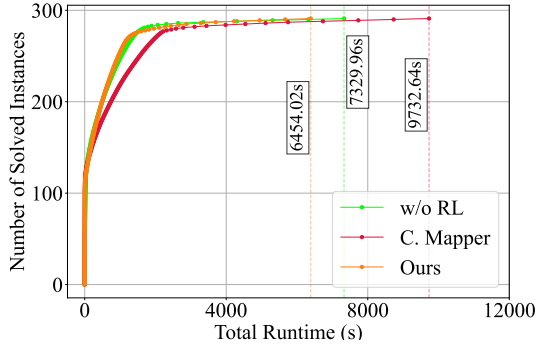


Fig. 5: Ablation Study

According to Figure 4, our proposed circuit-based preprocessing framework can significantly reduce runtime. To be specific, solving these 300 instances necessitates 19,422.38 seconds for the CaDiCaL Baseline pipeline and 11,073.88 seconds for the comparative pipeline (Comp.). In contrast, our method completes the task in a total of 7,179.80 seconds, marking a reduction of 63.03% and 35.16% in solving time compared to Baseline and Comp., respectively.

C. Ablation Studies

In the following experiments, we investigate the effectiveness of our RL-based agent and cost-customized mapper. We keep the same experiment environments and test cases above.

1) *Effectiveness of Logic Synthesis Agent*: To investigate the effectiveness of the RL agent, we conduct an ablation study by introducing another agent that randomly selects LS operations for $T = 10$ times (noted as w/o RL). The original setting with RL agent is noted as Ours. We test these two settings and summarized the results in Figure 5.

We observe that although the agent with a random policy is capable of employing LS operations to simplify the circuit, it still falls short in terms of achieving optimal solving time reduction. To be specific, solving these 300 instances necessitates 7329.96 seconds for the w/o RL and 6454.02 seconds for our w/ RL agent, marking a reduction of 11.95%.

2) *Effectiveness of Cost-Customized Mapper*: To showcase the effectiveness of our proposed cost-customized mapper,

we replace our cost-customized mapper with a mapper with conventional cost metrics. We denote the new setting as C. Mapper and our original setting as Ours. It is worth noting that both settings utilize the same RL agent with same parameters, and thus, we disregard the agent runtime ($\mathcal{T}_{agent}$) in the reported results.

As shown in Figure 5, the 300 instances processed by the conventional mapper take 9732.64 seconds to solve, which is 50.80% longer than the solving time of Our Mapper. We attribute this difference to the divergent optimization objectives: the conventional mapper minimizes area and delay, while our cost-customized mapper targets solving complexity directly.

V. CONCLUSION AND FUTURE WORK

This paper introduces a novel circuit-based preprocessing framework for circuit satisfiability problem, integrating logic synthesis with reinforcement learning to transform circuits into solver-friendly formats. Key innovations include an RL-guided logic synthesis pipeline for optimizing circuit transformations and a cost-customized LUT mapping strategy that minimizes branching complexity during SAT solving. Experiments on industrial benchmarks demonstrate significant runtime reductions—up to 63% compared to baseline approaches—while maintaining compatibility with state-of-the-art solvers. Ablation studies validate the effectiveness of both the RL agent and cost-customized mapper in improving solving efficiency.

This work highlights the potential of combining EDA innovations with machine learning to advance SAT-solving methodologies. Beyond CSAT, the proposed framework offers a promising direction for tackling other SAT variants and related combinatorial optimization problems, enabling more efficient and scalable solutions. Future work will explore extending this approach to sequential circuits and broader EDA applications.

ACKNOWLEDGMENTS

This work was partly supported by the General Research Fund of the Hong Kong Research Grants Council (RGC) under Grant No. 14212422 and 14202824 and partly by the National Technology Innovation Center for EDA.

REFERENCES

- [1] S. A. Cook, “The complexity of theorem-proving procedures,” in *Proceedings of the 3rd ACM symposium on Theory of computing*, 1971, pp. 151–158.
- [2] N. Sorensson and N. Een, “Minisat v1. 13-a sat solver with conflict-clause minimization,” *Theory and Applications of Satisfiability Testing – SAT 2005*, vol. 2005, no. 53, pp. 1–2, 2005.
- [3] A. Biere, K. Fazekas, A. Fleury, and M. Heisinger, “Cadical, kissat, paracooba, plingeling and treengeling entering the sat competition 2020,” *SAT COMPETITION*, vol. 2020, p. 50, 2020.
- [4] R. Ostrowski, É. Grégoire, B. Mazure, and L. Sais, “Recovering and exploiting structural knowledge from cnf formulas,” in *Principles and Practice of Constraint Programming*. Springer, 2002, pp. 185–199.
- [5] N. Eén and A. Biere, “Effective preprocessing in sat through variable and clause elimination,” in *Theory and Applications of Satisfiability Testing – SAT 2005*. Springer, 2005, pp. 61–75.
- [6] S. Subbarayan and D. K. Pradhan, “Niver: Non-increasing variable elimination resolution for preprocessing sat instances,” in *Theory and Applications of Satisfiability Testing – SAT 2005*. Springer, 2005, pp. 276–291.
- [7] C. Condrat and P. Kalla, “A gröbner basis approach to cnf-formulae preprocessing,” in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2007, pp. 618–631.
- [8] N. Eén, A. Mishchenko, and N. Sörensson, “Applying logic synthesis for speeding up sat,” in *Theory and Applications of Satisfiability Testing – SAT 2007*. Springer, 2007, pp. 272–286.
- [9] A. Hosny, S. Hashemi, M. Shalan, and S. Reda, “Drills: Deep reinforcement learning for logic synthesis,” in *Proceedings of the 25th Asia and South Pacific Design Automation Conference*. IEEE, 2020, pp. 581–586.
- [10] K. Zhu, M. Liu, H. Chen, Z. Zhao, and D. Z. Pan, “Exploring logic optimizations with reinforcement learning and graph convolutional network,” in *2020 ACM/IEEE 2nd Workshop on Machine Learning for CAD*, 2020, pp. 145–150.
- [11] Z. Shi, H. Pan, S. Khan, M. Li, Y. Liu, J. Huang, H.-L. Zhen, M. Yuan, Z. Chu, and Q. Xu, “Deepgate2: Functionality-aware circuit representation learning,” in *Proceedings of the 2023 IEEE/ACM international conference on Computer-aided design*, 2023.
- [12] R. J. Francis, J. Rose, and K. Chung, “Chortle: A technology mapping program for lookup table-based field programmable gate arrays,” in *Proceedings of the 27th ACM/IEEE Design Automation Conference*, 1991, pp. 613–619.
- [13] M. Soeken, H. Rienner, W. Haaswijk, E. Testa, B. Schmitt, G. Meuli, F. Mozafari, S.-Y. Lee, A. T. Calvino, D. S. Marakkalage *et al.*, “The epfl logic synthesis libraries,” *arXiv preprint arXiv:1805.05121*, 2018.
- [14] J. Marques-Silva, I. Lynce, and S. Malik, “Conflict-driven clause learning sat solvers,” in *Handbook of satisfiability*. IOS press, 2021, pp. 133–182.
- [15] G. Audemard and L. Simon, “Predicting learnt clauses quality in modern sat solvers,” in *Proceedings of the 21st International Joint Conference on Artificial Intelligence*, 2009, pp. 399–404.
- [16] M. W. Moskiewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik, “Chaff: Engineering an efficient sat solver,” in *Proceedings of the 38th ACM/IEEE Design Automation Conference*, 2001, pp. 530–535.
- [17] Z. Shi, M. Li, S. Khan, H.-L. Zhen, M. Yuan, and Q. Xu, “Satformer: Transformers for sat solving,” in *Proceedings of the 2023 IEEE/ACM international conference on Computer-aided design*, 2023.
- [18] R. Béjar and F. Manyà, “Solving the round robin problem using propositional logic,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 17, 2000, pp. 262–266.
- [19] C.-A. Wu, T.-H. Lin, C.-C. Lee, and C.-Y. Huang, “Qutesat: a robust circuit-based sat solver for complex circuit structure,” in *Proceedings Design, Automation and Test in Europe. Conference and Exhibition 200*. IEEE, 2007, pp. 1–6.
- [20] H.-T. Zhang, J.-H. R. Jiang, and A. Mishchenko, “A circuit-based sat solver for logic synthesis,” in *Proceedings of the 2021 IEEE/ACM international conference on Computer-aided design*, 2021, pp. 1–6.
- [21] M. Li, Z. Shi, Q. Lai, S. Khan, S. Cai, and Q. Xu, “On eda-driven learning for sat solving,” in *Proceedings of the 60th ACM/IEEE Design Automation Conference*. IEEE, 2023, pp. 1–6.
- [22] M. R. Berkelaar and J. A. Jess, “Technology mapping for standard-cell generators,” in *Proceedings of the 1988 IEEE/ACM international conference on Computer-aided design*, 1988, pp. 470–473.
- [23] L. Chen, Y. Chen, Z. Chu, W. Fang, T.-Y. Ho, R. Huang, Y. Huang, S. Khan, M. Li, X. Li *et al.*, “Large circuit models: opportunities and challenges,” *Science China Information Sciences*, vol. 67, no. 10, p. 200402, 2024.
- [24] M. Li, S. Khan, Z. Shi, N. Wang, H. Yu, and Q. Xu, “Deepgate: Learning neural representations of logic gates,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 667–672.
- [25] Z. Shi, Z. Zheng, S. Khan, J. Zhong, M. Li, and Q. Xu, “Deepgate3: Towards scalable circuit representation learning,” in *Proceedings of the 2024 IEEE/ACM international conference on Computer-aided design*, 2024.
- [26] Z. Shi, M. Li, S. Khan, L. Wang, N. Wang, Y. Huang, and Q. Xu, “Deeppti: Test point insertion with deep reinforcement learning,” in *2022 IEEE International Test Conference (ITC)*. IEEE, 2022, pp. 194–203.
- [27] A. Biere, “Aiger (aiger is a format, library and set of utilities for and-inverter graphs (aigs)),” <https://fmv.jku.at/aiger/>, 2006.
- [28] A. Mishchenko, S. Chatterjee, and R. Brayton, “Dag-aware aig rewriting a fresh look at combinational logic synthesis,” in *Proceedings of the 43rd ACM/IEEE Design Automation Conference*, 2006, pp. 532–535.
- [29] R. K. Brayton, “The decomposition and factorization of boolean expressions,” *International Symposium on Computer Architecture (ISCA 1982)*, pp. 49–54, 1982.
- [30] J. Cortadella, “Timing-driven logic bi-decomposition,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 22, no. 6, pp. 675–685, 2003.
- [31] H. Sato, Y. Yasue, Y. Matsunaga, and M. Fujita, “Boolean resubstitution with permissible functions and binary decision diagrams,” in *Proceedings of the 27th ACM/IEEE Design Automation Conference*, 1991, pp. 284–289.
- [32] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, “Human-level control through deep reinforcement learning,” *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [33] J. H. Liang, V. Ganesh, P. Poupart, and K. Czarnecki, “Learning rate based branching heuristic for sat solvers,” in *Theory and Applications of Satisfiability Testing – SAT 2016*. Springer, 2016, pp. 123–140.
- [34] H. Haanpää, M. Järvisalo, P. Kaski, and I. Niemelä, “Hard satisfiable clause sets for benchmarking equivalence reasoning techniques,” *Journal on Satisfiability, Boolean Modeling and Computation*, vol. 2, no. 1–4, pp. 27–46, 2006.
- [35] J. M. Dudek, K. S. Meel, and M. Y. Vardi, “The hard problems are almost everywhere for random cnf-xor formulas,” in *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, 2017, pp. 600–606.
- [36] A. Ling, D. P. Singh, and S. D. Brown, “Fpga technology mapping: a study of optimality,” in *Proceedings of the 42nd ACM/IEEE Design Automation Conference*, 2005, pp. 427–432.
- [37] A. Mishchenko *et al.*, “Abc: A system for sequential synthesis and verification,” URL <http://www.eecs.berkeley.edu/alanmi/abc>, vol. 17, 2007.
- [38] M. J. Heule, M. Iser, M. Järvisalo, and M. Suda, “Proceedings of sat competition 2024: Solver, benchmark and proof checker descriptions,” 2024.
- [39] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt, “Cadical 2.0,” in *International Conference on Computer Aided Verification*. Springer, 2024, pp. 133–152.